Q1. HTML -> JS -> HTML

```
<body>
    <input id="item-name" type="text" placeholder="item name">
    <input id="item-price" type="text" placeholder="item price">
    <button id="earning" type="submit">EARNING</button>
    <div id="feedback">MY BUDGET:</div>

    <script src="index.js"></script>
</body>

// to persist across multiple button clicks, we need global variables
let earning_items=[]
let total_earnings=0

document.getElementById("earning").addEventListener("click", function()) {

       //INPUT TAKING================================
       let item  = document.getElementById("item-name").value
       let price = Number(document.getElementById("item-price").value)

       // CALCULATION================================
       total_earnings += new_earning_price
       earning_items.push(new_earning_item)

       // SHOW IN HTML============================
       let feedback = ""            // get  some feedback
       let out = "Attempts: " + attempt + " | Feedback: " + feedback
       document.getElementById("output").innerHTML = out
}
```

SQL

```
SELECT column_name, AGGREGATE_FUNCTION(column)
FROM table_name
WHERE row_condition
GROUP BY column_name
HAVING aggregate_condition
ORDER BY column_name DESC
LIMIT n;
```

Q2. HTML(form) -> PHP

```
<body>
    <form action="index.php" method="post">
        <input name="numOfStudents" placeholder="Num of students">
        <input name="slicePerStudent" placeholder="Slice per student">
        <input name="slicePerPizza" placeholder="Slice per pizza">
        <button type="submit">Submit</button>
    </form>
</body>

<?php
// Form input collection ==========================
$numOfStudents = (int) $_POST["numOfStudents"];
$slicePerStudent = (int) $_POST["slicePerStudent"];
$slicePerPizza = (int) $_POST["slicePerPizza"];

// Calculation ====================================
pizzaNeeded = ceil(slicePerStudent * $numOfStudents / $slicePerPizza);
leftOverSlices = (pizzaNeeded * slicePerPizza) − (slicePerStudent *
$numOfStudents);
$wastedMoney = (1050 / $slicePerPizza) * $leftOverSlices;

// Feedback ========================================
echo "Total Pizzas: $pizzaNeeded
        | LeftOver Slices: $leftOverSlices
        | Wasted Money(BDT): $wastedMoney";

?>
```

Syntax

Updating Text: .innerHTML (if you want to include <br> or spans)
Disabling Elements: .disabled = true to lock inputs or buttons
Regex (.test): /[A-Z]/.test(string) ==> Check upperCase
/[a-z]/.test(string) ==> Check lowerCase
/[0-9]/.test(string) ==> Check number
/[@#$]/.test(string) ==> Check SpecialCharacter
Math.random: Math.floor(Math.random() * (Max - Min + 1)) + Min
php* $attendees = (int) $_POST["attendees"];
js* let price = Number(document.getElementById("item-price").value)

Q3. PHP + MySQL

```
CREATE DATABASE exam_practice;
USE exam_practice;

CREATE TABLE products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10,2),
    Quantity INT,
    Rating DECIMAL(3,1)
);

INSERT INTO products VALUES
(1, 'Laptop', 'Electronics', 70000, 5, 4.8),
(2, 'Mouse', 'Electronics', 1500, 15, 4.2),
(3, 'Chair', 'Furniture', 8000, 8, 4.3),
(4, 'Desk', 'Furniture', 12000, 6, 4.6),
(5, 'Bottle', 'Accessories', 500, 20, 4.1),
(6, 'Pen', 'Accessories', 100, 25, 4.7);

<?php
// CONNECTION=====================================
$conn = new mysqli("localhost","root","","db_name");
if($conn->connect_error==false)
    echo "connection successful<br>";

// Selection query================================
$sql = "SELECT * FROM products";
$result = conn –> query(sql);
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "Course: " . $row["CourseTitle"] .
        " | Enrolled: " . $row["total_students"] . "<br>";
    }
}

// Update query===================================
$sql="UPDATE student_final
      SET Grade=Grade+5
      WHERE Grade>80 AND Grade+5<=90 ";
$result = conn –> query(sql);

// Connection close===============================
$conn->close();

?>
```

DB

```
-- 1. Count records by category/grade/status
SELECT Category, COUNT(*) AS total
FROM table_name
GROUP BY Category;

-- 2. Count groups, show only those above a limit
SELECT Status, COUNT(*) AS total
FROM table_name
GROUP BY Status
HAVING COUNT(*) > 1;

-- 3. Calculate total per group + sort descending
SELECT Category, SUM(Revenue) AS total
FROM table_name
GROUP BY Category
ORDER BY SUM(Revenue) DESC;

-- 4. Multiple aggregate calculations per group
SELECT Category, COUNT(*) AS total,
       AVG(Rating) AS avg_rating,
       SUM(Visitors) AS total_visitors
FROM table_name
GROUP BY Category
HAVING AVG(Rating) > 4.4
ORDER BY AVG(Rating) DESC;

-- 5. Update records matching multiple conditions
UPDATE table_name
SET Status = 'Grace Period', Fee = 0
WHERE Status = 'Overdue' AND DaysOverdue < 7;

-- 6. Increase/decrease by percentage (10%) => 100+10 => 110% => 1.10
UPDATE table_name
SET Price = Price * 1.10
WHERE Price > 500;

-- 7. Update only if the new value stays within a limit
UPDATE table_name
SET Salary = Salary + 5000
WHERE Salary > 50000 AND Salary + 5000 <= 60000;

-- 8. Compare against the average of a group
SELECT ProductName,
       CASE
           WHEN Revenue > (
               SELECT AVG(Revenue)
               FROM sales_data
               WHERE CategoryID = s.CategoryID
           ) THEN 'Top Seller'
           ELSE 'Regular Seller'
       END AS Label
FROM sales_data s;
```